COS 214 Final Report

TIVIDY: Workflow Management System

November 02, 2026



https://github.com/u21450481-art/214FinalProjectTIVIDY

1. Contribution Statement

This project was a collaborative effort. We all came together and had two big meetings on campus in order to come up with the overall design of our system. We decided on the foundational architecture and core system design together during extensive in-person planning sessions on campus. Furthermore, all team members equally contributed to the Doxygen documentation.

Individual contributions:

Ms. Carla Erasmus (Team leader)

- Organized meetings, delegated tasks, and kept the project on track.
- Implemented the Builder, Facade, and Prototype patterns.
- Built the core text user interface (TUI) and created the first draft of the main UML diagram.

Miss. Bowdene Pillay

- Implemented the Mediator and Strategy patterns and wrote their unit tests.
- Created the PowerPoint presentation for our final live demo.
- Did the CI- pipeline for bonus marks.

Miss. Vimbai Chivunga

- Implemented the State and Chain of Responsibility patterns, and wrote their unit tests.
- Added the Chain of Responsibility logic into the TUI.
- Created the UML State and Sequence diagrams.

Miss. Tshegofatso Mautjane

- Implemented two Composite patterns, the Iterator, and the Factory Method, and wrote unit tests.
- Set up the Docker configuration for the project.
- Improved the layout and design of our main UML diagram.

Miss. Luthando Shilhwaio

- Implemented the Command and Adapter patterns and wrote their unit tests.
- Wrote additional unit tests for the Builder, Facade, and Prototype patterns.
- Helped create the UML Activity diagrams and finalized the main UML diagram for submission.

2. Workflow Research

Workflows solve inefficiency by turning tasks into a repeatable and organized sequence of steps. Workflows are implemented into workflow management systems which then provide the software to automate and manage these tasks.

Workflows provide the blueprint of a task approach. As tasks begin to happen workflow instances are created. Workflow instances provide a single, active occurrence of a defined workflow process, with its own user data and active tasks.

A workflow instance begins with a trigger. Once an event is triggered, a routing pattern or a combination of such is decided upon i.e sequential, parallel or conditional routing. This is how work moves from one activity to the next. Tasks are then assigned to the correct person using a role-based, direct, conditional etc. assignment approach. As tasks flow from person to person, data is collected and sent to the next person without the need for manual data flow between steps. Connections to other tools through API updates, built-in connectors, or webhooks help to integrate helpful systems into your workflow.

Workflow activities describe what needs to be done, defined by a person or automated by software. When a workflow instance reaches an activity, a work item is created, a single unit of work that is assigned and tracked using statuses (created, assigned, in progress, or completed). Activities are often represented as rounded rectangles, events as circles and gateways as diamonds. Whilst work items appear at runtime rather than on a diagram.

Approvals (single, parallel etc.) are a stored outcome on a work item, conditional gateways read this outcome and decide the next step. Rejections are also outcomes that can be dealt with by looping back, terminating or choosing an alternative path. Escalations handle work that is late or stuck. Deadlines are attached to work items and when they are overdue defined actions are taken (reminder, reassign work etc.).

Dependencies define the sequence of operations and can be handled through sequence flows, parallel joins, data dependencies etc. Events happen during and outside processes and are detected, assigned the correct process and then reacting to them. Technical and business failures can be handled by retries, error paths, or manual intervention.

Workflow engines are reusable across different processes because the engine only clarifies the definitions and doesn’t know the specifics of the processes. Every new definition stems from a new process and not a new engine code. Real workflow systems model nested activities/sub-processes

through work hierarchies that consist of stages, tasks and sub-processes. A reusable process called BPMN call activity is what a sub-process is rooted as. Changes to workflow definitions are handled while work is already in progress by the definitions being versioned. By doing this the instances running stay on the version they started with instead of the latest willing is what the new instances use.

References:

1. Sam, A.E. (2026). *What is a workflow system and how does it work? | Decode - A publication by Zoho Creator*. [online] Decode - A publication by Zoho Creator. Available at: https://www.zoho.com/creator/decode/workflow-system-how-it-works [Accessed 4 Oct. 2026].
2. Atlassian (n.d.). *Workflow Management: Strategies & Best Practices*. [online] Atlassian. Available at: https://www.atlassian.com/agile/project-management/workflow-management [Accessed 4 Oct. 2026].
3. Hyperbots.com. (2025). *What is Approval Escalation Workflow?* [online] Available at: https://www.hyperbots.com/glossary/approval-escalation-workflow [Accessed 4 Oct. 2026].
4. Camunda.io. (2025). *Workflow patterns | Camunda 8 Docs*. [online] Available at: https://docs.camunda.io/docs/components/concepts/workflow-patterns/ [Accessed 4 Oct. 2026].

3. Two industry scenarios

Retail returning an item:

Industry: Retail. A store accepts an item that a customer brings back, decides whether the return is valid, and refunds the customer.

This system automates customer item returns,eligibility validation according to the item condition. We start an instance with the builder and facade design patterns.The mediator controls the workflow state changes(NotStarted->InProgress->Complete),utilizing the strategy pattern to assign tasks to available employees.The Command logs every decision into history allowing for undo and redo in case of mistakes or cancels.The return information is stored in a legacy system as a receipt which is then adapted into data the tividy engine can process into payload.Unlike the Insurance Claim scenario, the retail return works on single-point validation such as checking item condition and receipt authenticity.

Process an insurance claim:

Industry: Insurance.
A customer makes an insurance claim and the insurance company records the claim information, assesses the damage, resolves the issue, approves the payout and does the settlement.

When a customer wants to make a claim an insurance claim workflow is instantiated by the facade and builder. A claims adapter is used to adapt the customer's information into the standard format used by the company payload. A task tree is then constructed which contains all the tasks and subprocesses that occur in the workflow. Each task is assigned a participant that handles its execution by the mediator using the assignment strategy pattern by the mediator. The mediator also controls the execution of the tasks and the state changes of each task. The process is stored into history using the Command pattern to support undo and redo operations.

4. Design and UML Documentation

GoF Design Patterns

Building a workflow

Facade

Role: Gives the client/TUI one simple entry point (WorkflowEngine) to the whole system, so it doesn't need to know how the other classes fit together.

Justification: The intent is to provide a unified, higher-level interface to a set of subsystem interfaces. Starting a workflow touches the builder, the prototype, the mediator and the strategy, and the facade hides all that behind one call.

Participant mapping:

- Subsystem classes: Director and CustomDefinitionBuilder, WorkflowDefinition, WorkflowInstance, EngineMediator, the strategies, the adapters, CommandHistory
- Client: the TUI (main.cpp)



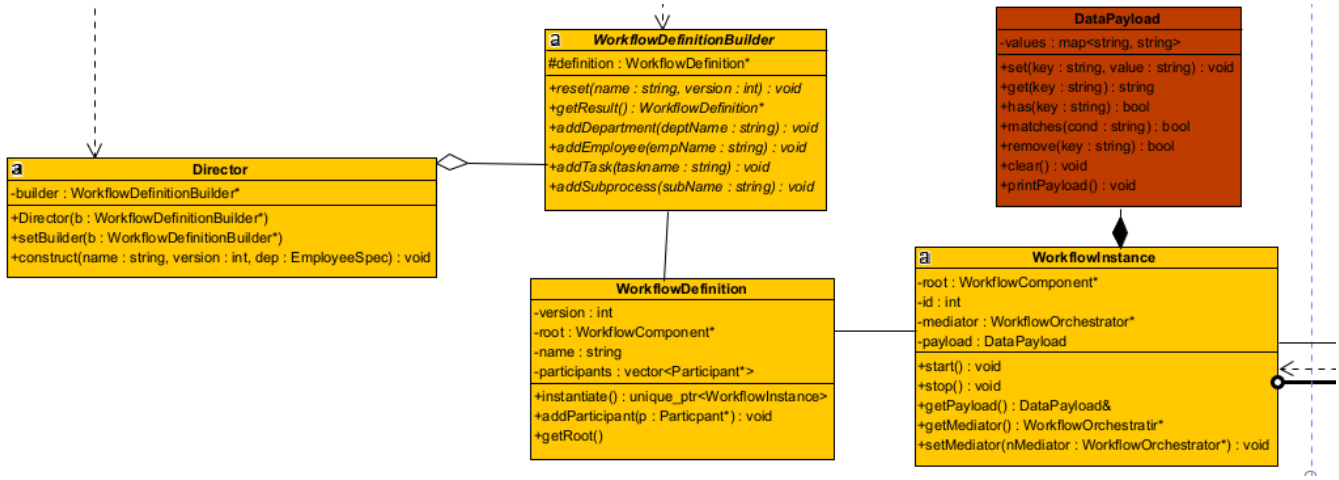
Builder

Role: Builds a WorkflowDefinition step by step (departments, employees, subprocesses, tasks).

Justification: The intent is to separate construction from representation, so the same process can create different representations. The same steps build a retail return and an insurance claim, and only the data differs. We didn't use Factory Method because it creates an object in one call, and a WorkflowDefinition has many parts that must be added in a valid order.

Participant mapping:

- Builder: WorkflowDefinitionBuilder
- ConcreteBuilder: CustomDefinitionBuilder
- Director: Director
- Product: WorkflowDefinition
- Client: WorkflowEngine



Prototype

Role: Creates a new instance by copying a WorkflowDefinition's task tree with clone().

Justification: The intent is to create new objects by copying a prototypical instance. The WorkflowDefinition is the prototype, and each instance gets its own copy, so running it never changes the original. We didn't rebuild with the Builder each time, because that would repeat every step for every run.

Participant mapping:

- Prototype: WorkflowComponent (clone())
- ConcretePrototype: Task (copies itself) and Subprocess (copies itself and all its children)
- Client: WorkflowDefinition (instantiate()), called by WorkflowEngine

Structure

Iterator

Role: Walks through the task tree one item at a time.

Justification: The intent is to access the elements of an aggregate sequentially without exposing its representation. The facade lists all tasks, or only the completed ones, with the same loop and never sees how the tree is stored. A plain loop over the children wouldn't work, because it only reaches one level of a tree with phases inside phases.

Participant mapping:

- Iterator: WorkflowIterator
- ConcreteIterator: DepthFirstIterator (every node) and CompletedTaskIterator (only completed tasks)
- Aggregate: WorkflowComponent (createIterator())
- ConcreteAggregate: Subprocess
- Client: WorkflowEngine

Factory Method

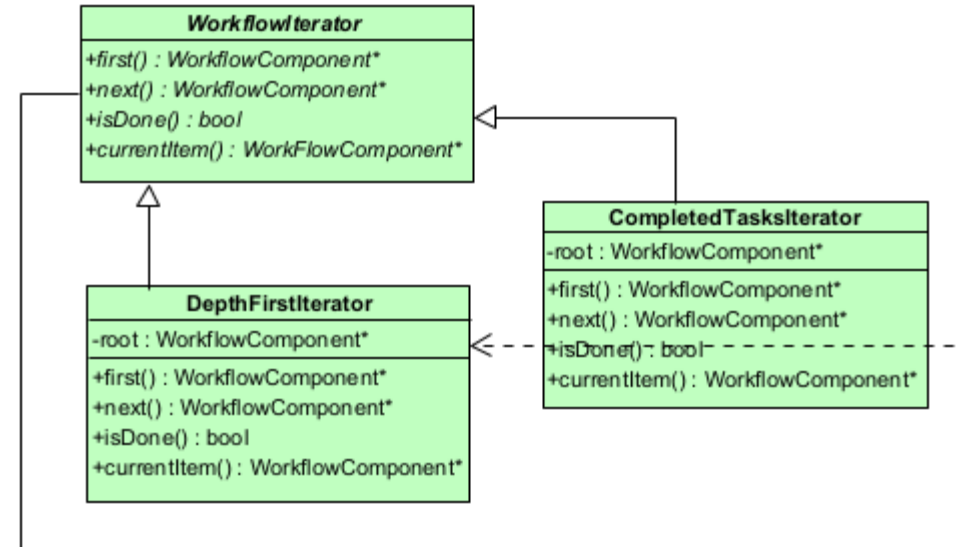
Role: Any node in the task tree can be asked for an iterator with createIterator(), and the node decides which iterator class to create.

Justification: The intent is to define an interface for creating an object but let subclasses decide which class to instantiate. The facade just asks a node for an iterator and never names DepthFirstIterator, so a new way of walking the tree doesn't change the facade.

Participant mapping:

- Creator: WorkflowComponent (declares the factory method `createIterator()`)
- ConcreteCreators: Task and Subprocess (inherit it and could override it)
- Product: WorkflowIterator
- ConcreteProduct: DepthFirstIterator

- Client: WorkflowEngine (findNode walks the tree through createIterator())



Composite

Role:Treats a single task and a group of tasks the same way, so the whole workflow is one tree. The same goes for employees and departments.

Justification: The intent is to compose objects into trees so clients can treat individual objects and compositions uniformly. The mediator calls the same methods on a Task or a Subprocess without checking which it is. A flat list wouldn't work, because workflows have phases inside phases.

Participant mapping:

- Tasks: Component is WorkflowComponent, Composite is Subprocess, Leaf is Task
- People: Component is Participant, Composite is Department, Leaf is Employee
- Client: EngineMediator and WorkflowEngine

Lifecycles and Approvals

State

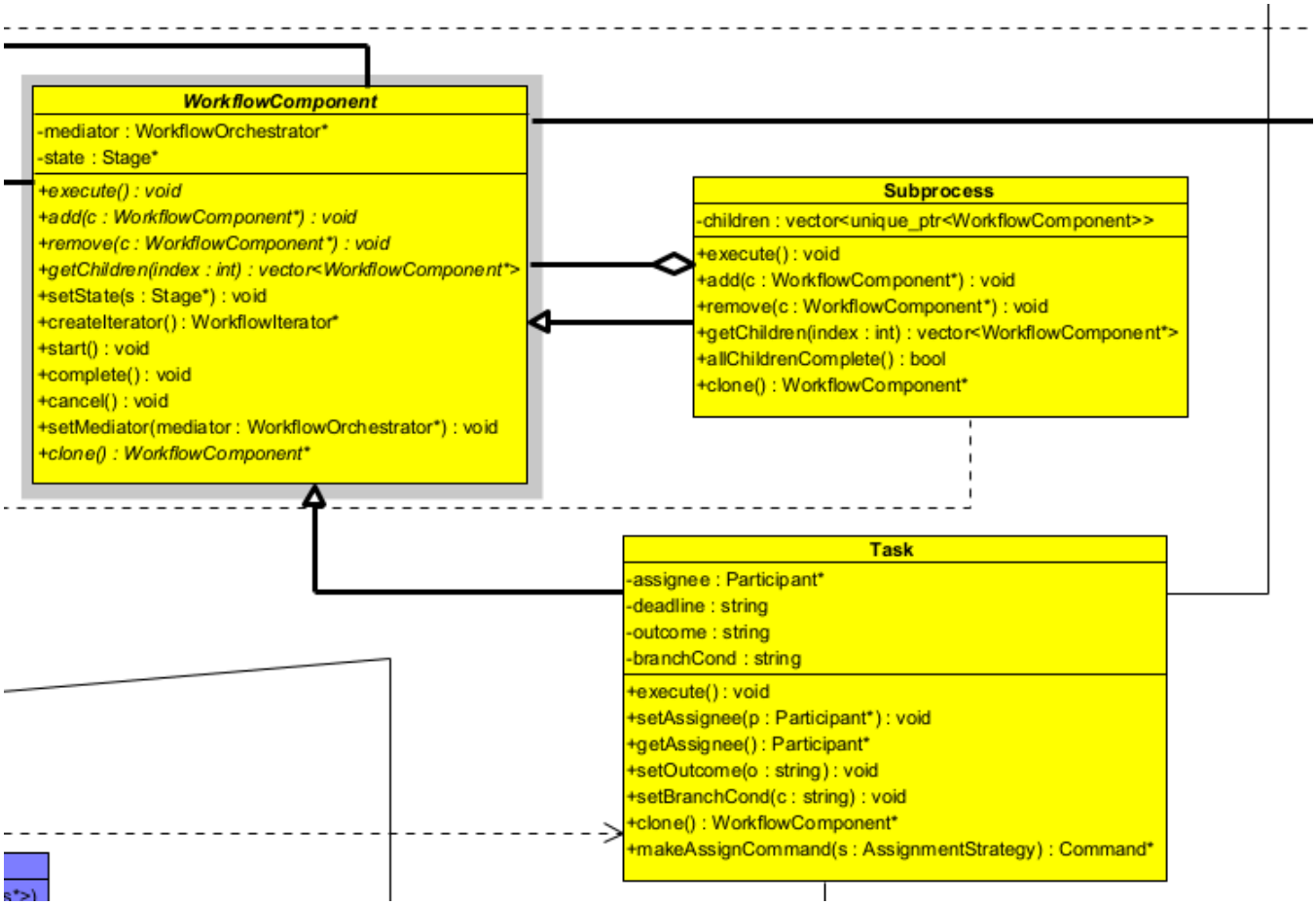
Role:Controls what a task is allowed to do at each stage: not started, in progress, complete, rejected or cancelled.

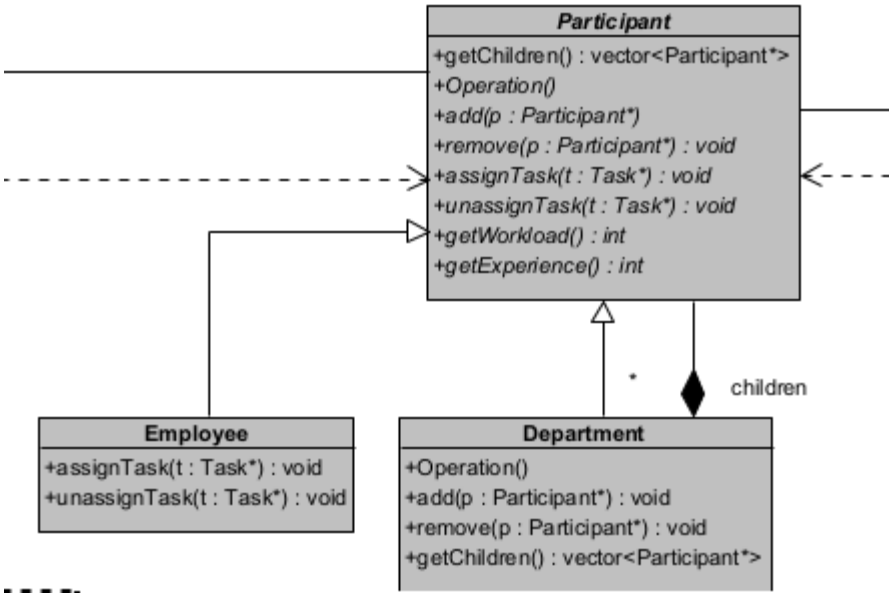
Justification: The intent is to let an object change its behaviour when its internal state changes. A task moves between stages by itself, and each stage decides which moves are legal. We didn't use Strategy

because a strategy is picked by the client from outside, whereas a state changes on its own as the work progresses.

Participant mapping:

- Context: WorkflowComponent (Task and Subprocess)
- State: Stage
- ConcreteStates: NotStarted, InProgress, Complete, Rejected, Cancelled (shared single objects)


WorkflowComponent
-mediator : WorkflowOrchestrator*
-state : Stage*
+execute() : void
+add(c : WorkflowComponent*) : void
+remove(c : WorkflowComponent*) : void
+getChildren(index : int) : vector<WorkflowComponent*>
+setState(s : Stage*) : void
+createIterator() : WorkflowIterator*
+start() : void
+complete() : void
+cancel() : void
+setMediator(mediator : WorkflowOrchestrator*) : void
+clone() : WorkflowComponent*
Subprocess
-children : vector<unique_ptr<WorkflowComponent>>
+execute() : void
+add(c : WorkflowComponent*) : void
+remove(c : WorkflowComponent*) : void
+getChildren(index : int) : vector<WorkflowComponent*>
+allChildrenComplete() : bool
+clone() : WorkflowComponent*
Task
-assignee : Participant*
-deadline : string
-outcome : string
-branchCond : string
+execute() : void
+setAssignee(p : Participant*) : void
+getAssignee() : Participant*
+setOutcome(o : string) : void
+setBranchCond(c : string) : void
+clone() : WorkflowComponent*
+makeAssignCommand(s : AssignmentStrategy) : Command*



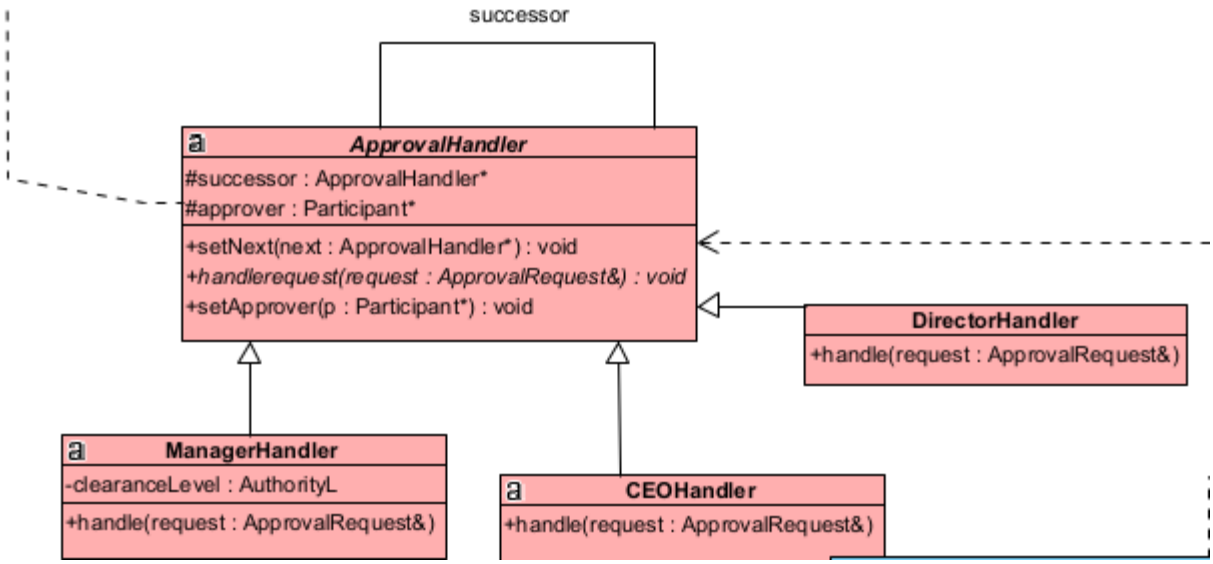
Chain of Responsibility

Role: Passes an approval request from Manager to Director to CEO until someone with enough authority handles it.

Justification: The intent is to avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle it. The sender doesn't know who will approve, and a bigger request just travels further up. We didn't use Strategy, because a strategy picks one rule up front, and here we can't tell in advance who is senior enough.

Participant mapping:

- Handler: ApprovalHandler
- ConcreteHandlers: ManagerHandler, DirectorHandler, CEOHandler
- Request: ApprovalRequest
- Client: EngineMediator (requestApproval())



Coordination

Mediator

Role: Decides what happens when a task reports an event, such as starting the next task or completing a group when all its tasks are done.

Justification: The intent is to encapsulate how a set of objects interact, so they don't refer to each other directly. Tasks only talk to the mediator, so one task never starts another directly, and the sequencing logic lives in one place.

Participant mapping:

- Mediator: WorkflowOrchestrator
- ConcreteMediator: EngineMediator
- Colleagues: WorkflowComponent (Task and Subprocess)

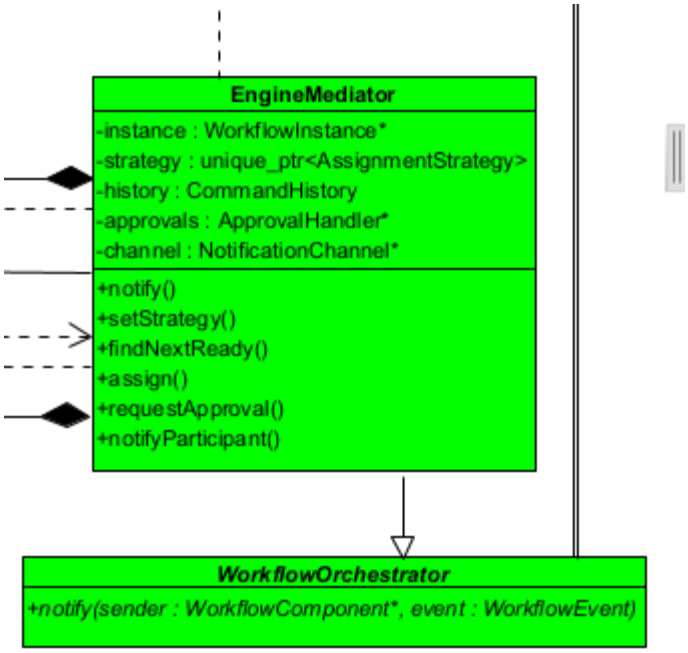
Strategy

Role: Chooses who a task is assigned to, using a rule that can be swapped.

Justification: The intent is to define a family of algorithms, encapsulate each one and make them interchangeable. LeastLoaded, RoleBased and MostExperience can be swapped at any time (the TUI has a button for it) without touching the mediator.

Participant mapping:

- Strategy: AssignmentStrategy
- ConcreteStrategies: LeastLoadedStrategy, RoleBasedStrategy, MostExperienceStrategy
- Context: EngineMediator



Actions

Command

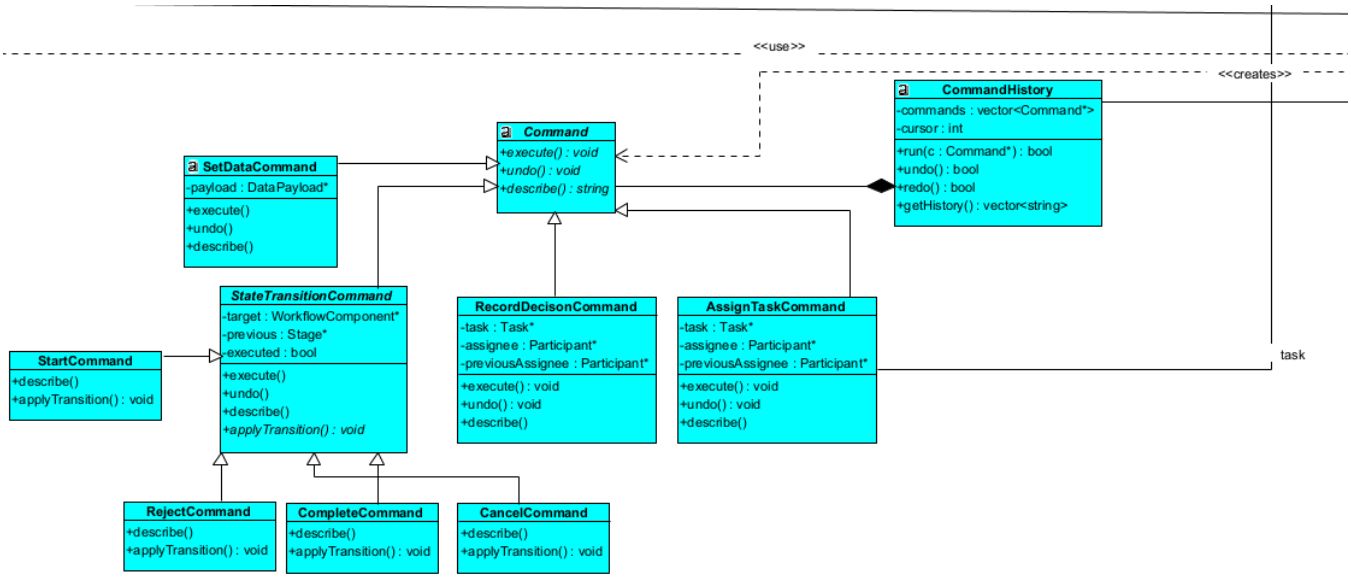
Role: Turns every action (start, complete, cancel, assign, set data, import) into an object that is run and recorded.

Justification: The intent is to encapsulate a request as an object so requests can be logged and undone. That gives us the history list, undo and redo, and a MacroCommand can group steps so they undo together. Calling methods directly would give no history and no undo, and a Memento would have to copy the whole instance, while a command only remembers what it changed.

Participant mapping:

- Command: Command

- ConcreteCommands: StartCommand, CompleteCommand, RejectCommand, CancelCommand (all through StateTransitionCommand), AssignTaskCommand, RecordDecisionCommand, SetDataCommand
- Invoker: CommandHistory
- Receiver: WorkflowComponent and Task, DataPayload
- Client: EngineMediator



The outside world

Adapter

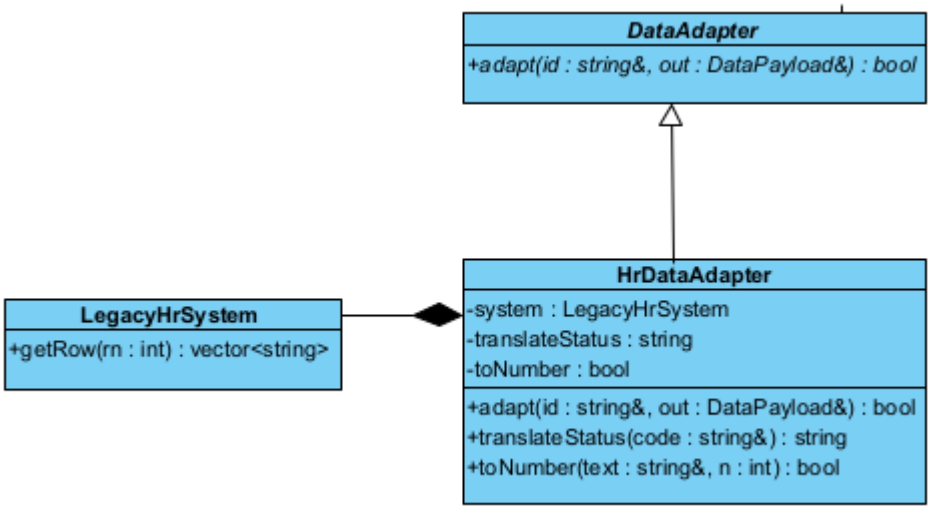
Role:Loads data from outside systems into our DataPayload using standard keys.

Justification:The intent is to convert one class's interface into the one clients expect. The HR, claims and returns systems each use their own format, and each adapter translates it into our standard keys,making it possible to process data stored in different formats easily.

Participant mapping:

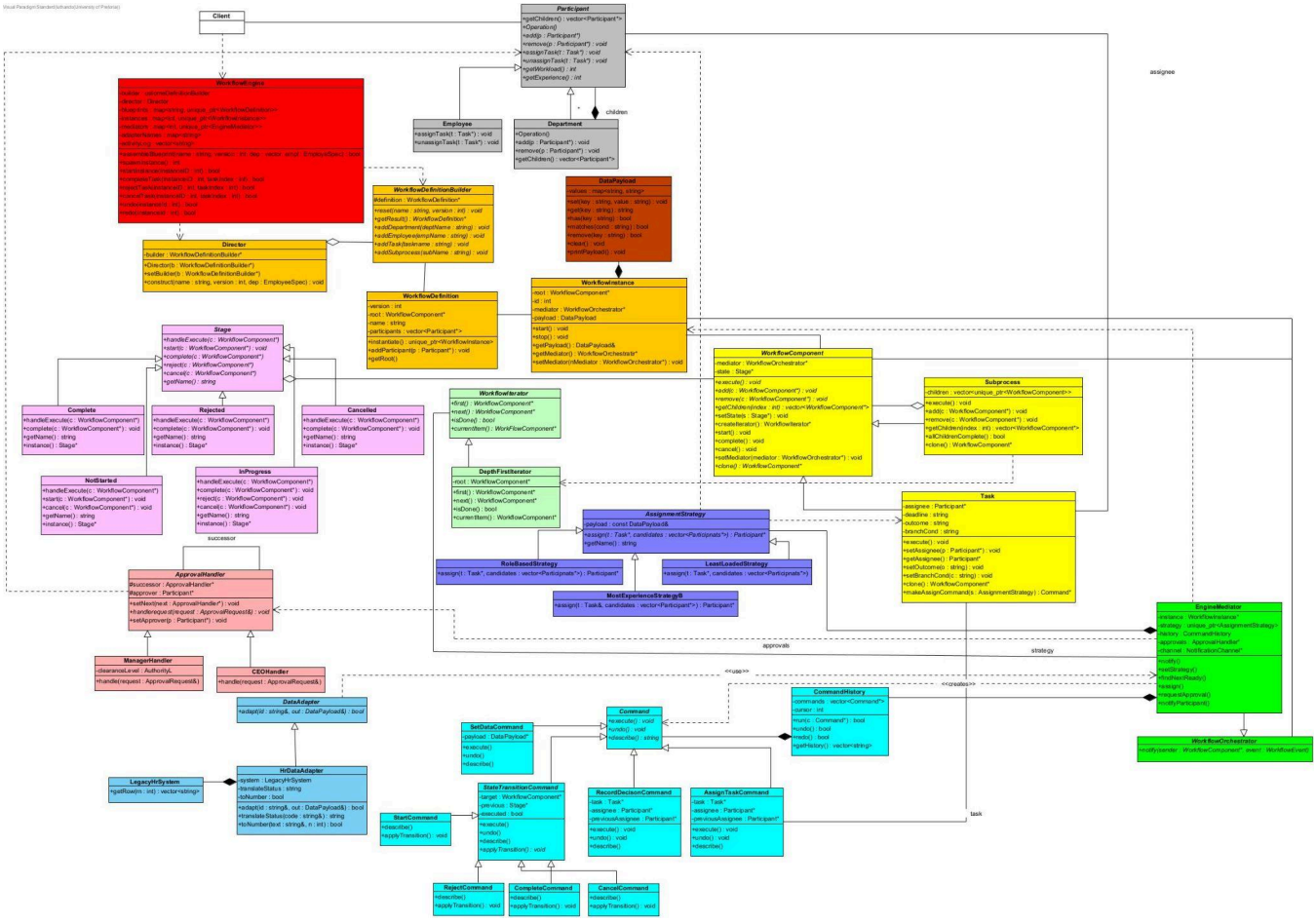
- Target: DataAdapter
- Adapters: HrDataAdapter, ClaimsDataAdapter, ReturnsDataAdapter
- Adaptees: LegacyHrSystem, ClaimsSystem, ReturnsSystem

- Client: EngineMediator (importData()), reached through WorkflowEngine

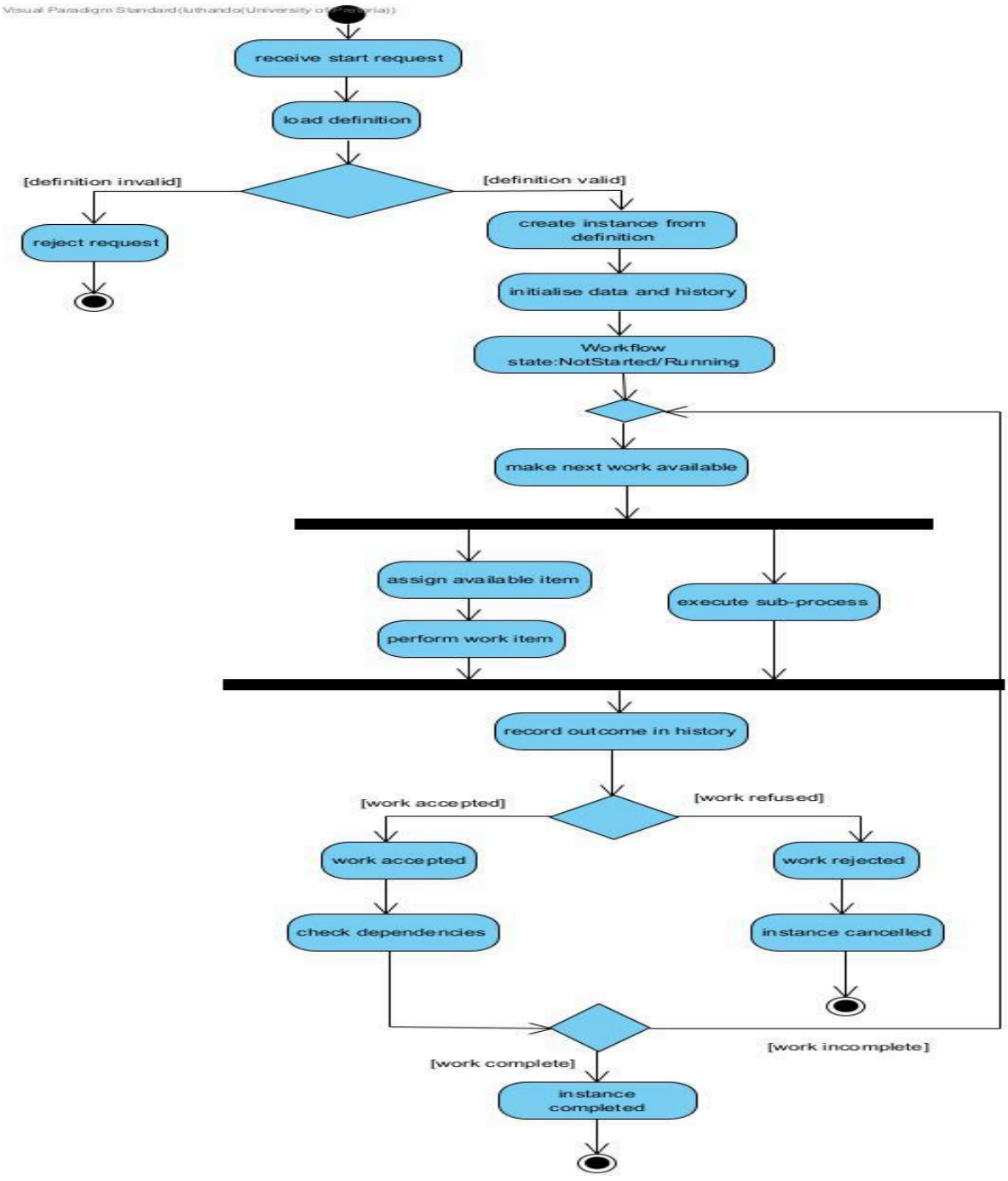


GoF Category	Design Patterns
Creational	Builder, Prototype, Factory Method
Structural	Facade, Composite, Adapter
Behavioral	Iterator, State, Chain of Responsibility, Mediator, Strategy, Command

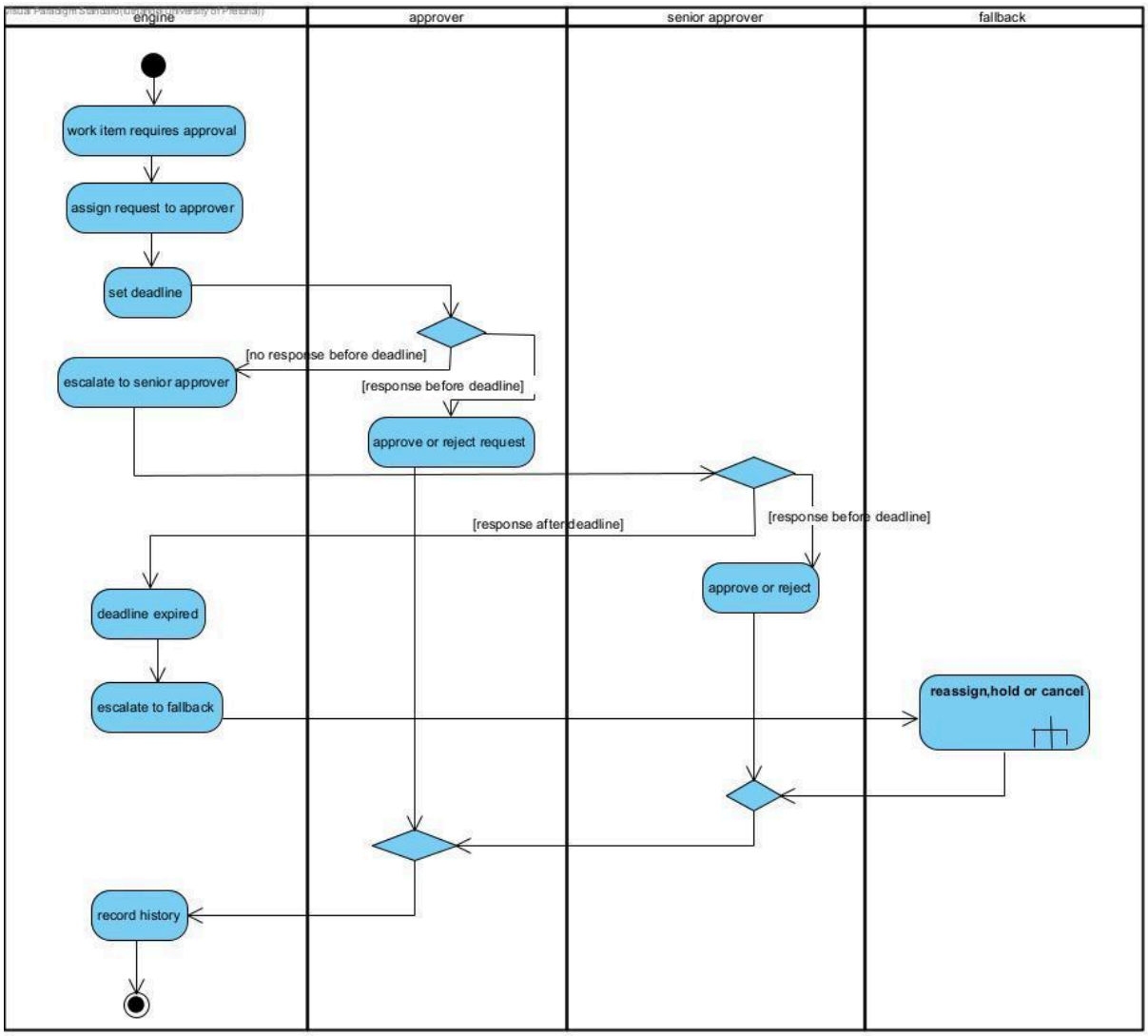
Diagrams


Client
Participant
WorkflowEngine
Employee
Department
children
assignee
WorkflowDefinitionBuilder
DataPayload
Director
WorkflowDefinition
WorkflowInstance
Stage
WorkflowComponent
Subprocess
Complete
Rejected
Cancelled
WorkflowIterator
DepthFirstIterator
NotStarted
InProgress
successor
Task
AssignmentStrategy
RoleBasedStrategy
LeastLoadedStrategy
MostExperienceStrategy
ApprovalHandler
ManagerHandler
CEOHandler
approvals
strategy
EngineMediator
<<use>>
<<creates>>
DataAdapter
HrDataAdapter
LegacyHrSystem
CommandHistory
Command
SetDataCommand
StartCommand
StateTransitionCommand
RecordDecisionCommand
AssignTaskCommand
RejectCommand
CompleteCommand
CancelCommand
task
WorkflowOrchestrator

AD1-generic workflow execution showing progression of a workflow instance from creation until completion



AD2:generic TIVIDY workflow, approval and escalation,

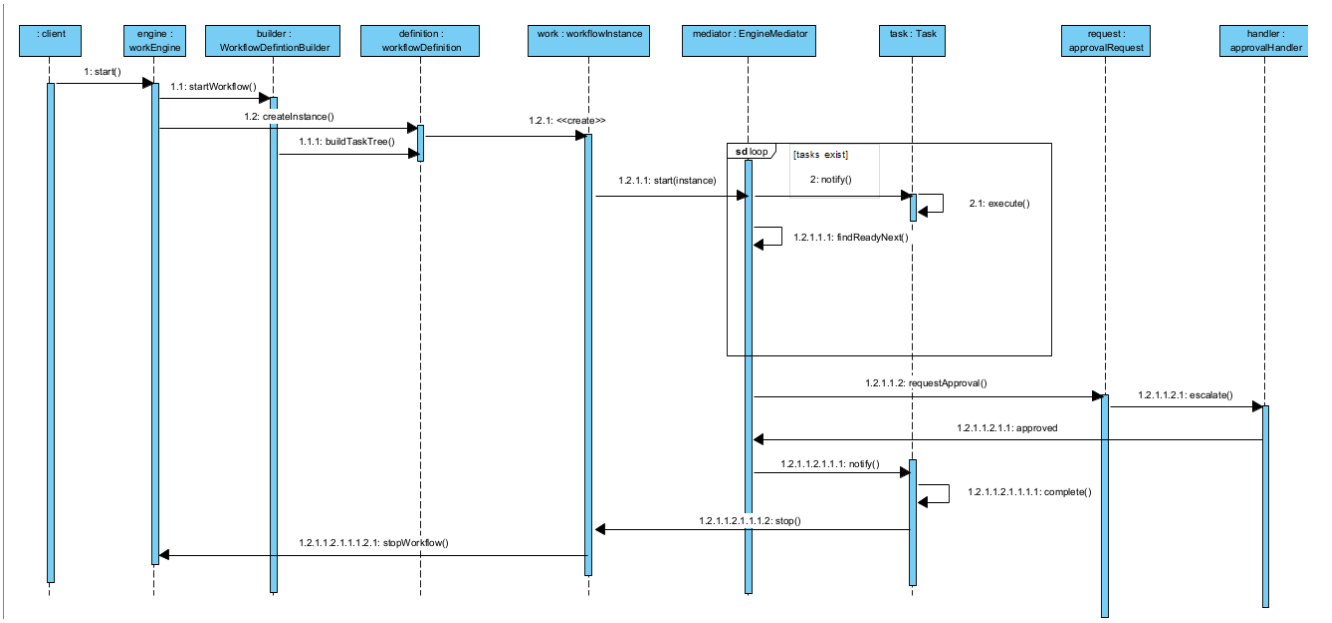


AD3: An important workflow from your selected scenario

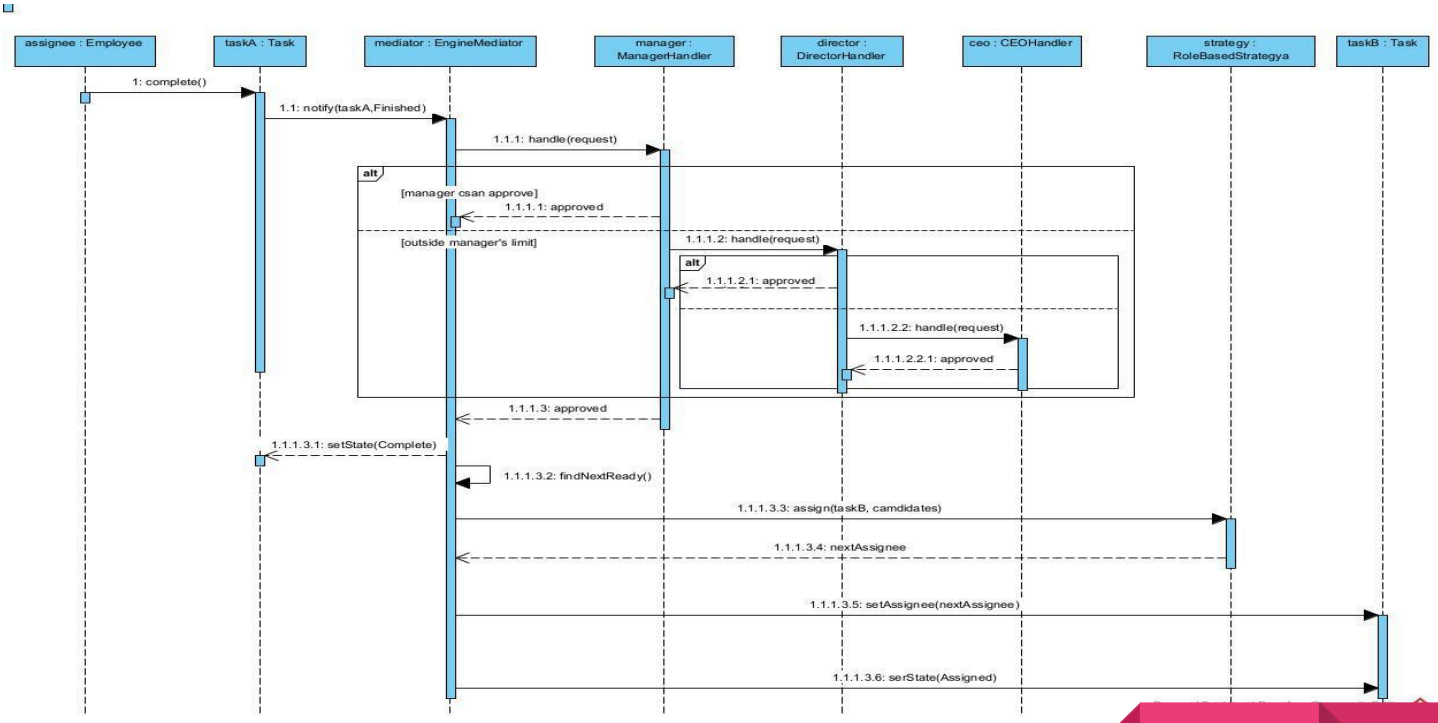


Task 6

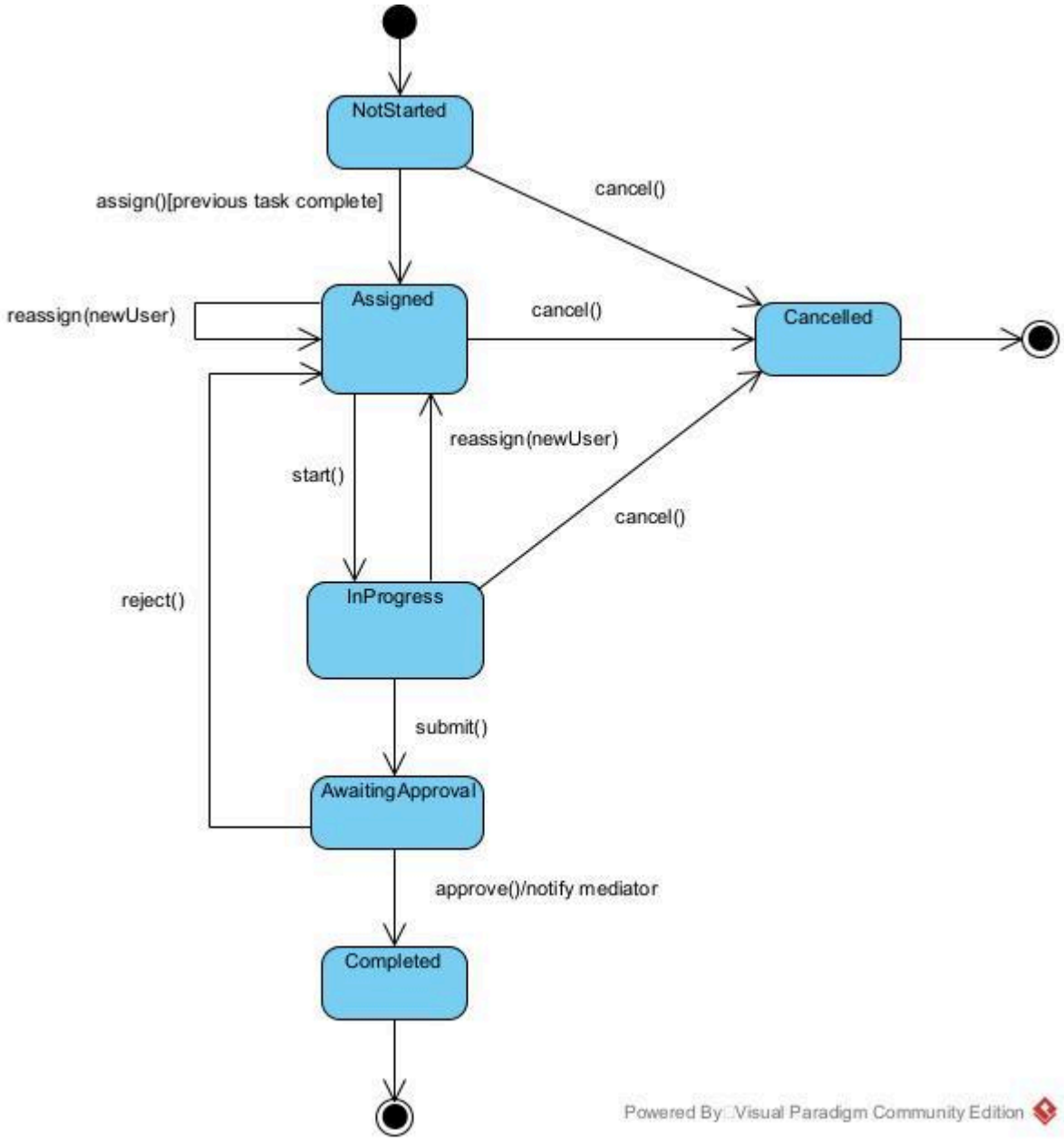
Sequence Diagram 1:



Sequence Diagram 2:



State Diagram:



Design decisions

1. Abstract builder carries real construction steps

Problem: The Director could only produce an empty workflow.

Original: The builder had reset(), buildParticipants() and buildTaskTree(). None of them took any data.

Change: Replaced them with reset(name, version), addDepartment, addEmployee, addSubprocess and addTask in the abstract builder.

Why: The Director only knows the abstract builder, so it needs those steps there to drive any builder without a downcast.

Diagram: Update the operations and parameters on WorkflowDefinitionBuilder.

2. One generic Director, no per-scenario Directors

Problem: Should each scenario (retail, insurance) get its own Director?

Original: One Director with a fixed three-step construct().

Change: One generic construct(name, version, departments, employees, subprocesses, tasks). The scenarios are only data in loadDemos.

Why: Our tutor wanted the engine to work for any industry. Separate Directors per scenario would hard-code industries into the engine. We also considered Audited and FastTrack Directors.

Diagram: Update construct on Director to show the full parameter list.

3. Spec structs in Director.h

Problem: How do we pass a whole workflow's data to the Director?

Original: The structs sat in the facade header, and the Director took loose parameters.

Change: EmployeeSpec, SubprocessSpec and TaskSpec live in Director.h.

Why: A struct keeps each item's fields together, so we don't need parallel lists that can go out of step. Putting them in Director.h means the Director doesn't depend on the facade.

4. Builder validates its input and stores branch conditions

Problem: The builder hid mistakes. A typo'd department made the employee standalone, duplicates overwrote each other, and the branch condition was logged but never saved.

Original: The constructor created an empty WorkflowDefinition, and failures were silent.

Change: The builder throws on a missing reset, empty or duplicate names (including ROOT) and unknown parents or departments. The condition is stored on the Task, and branchCondition now comes before deadline in addTask.

Why: Silent failure produced broken workflows that were hard to trace. The old parameter order stored the condition as the deadline without any warning.

Diagram: Update addTask and addEmployee signatures.

5. The facade assigns instance IDs

Problem: Each WorkflowDefinition numbered its own instances, so two blueprints both produced instance #1.

Original: instantiate() kept its own counter, and the facade rebuilt the instance with a setRoot(nullptr) workaround.

Change: instantiate(int id) takes the ID from the facade. Removed setRoot, setName and setVersion.

Why: IDs are a facade concern. Renaming a definition after it was registered would also have broken its lookup key.

Diagram: Change instantiate() to instantiate(id) on WorkflowDefinition.

6. The facade connects many subsystems

Problem: Our tutor pointed out that a facade only wrapping the builder isn't a facade.

Original: addCustomTask and similar wrappers around the builder.

Change: The facade now wires the Director and Builder, Prototype, Mediator, Strategy, Command history, Adapter, and Iterator. We removed the step-by-step wrappers, so the TUI collects lists and calls assembleBlueprint. Payload changes and imports go through the mediator so they can be undone. The TUI refers to tasks by row number, so it never holds a raw pointer.

Why: One call hides a whole sequence across several subsystems. Row numbers keep ownership of the trees inside the engine.

Diagram: Update WorkflowEngine: add getTaskRows, setStrategy, setAdapter, importData, setPayloadValue, getCompletedTasks, generateReport and exportReport.

7. Subsystems don't print to the terminal

Problem: std::cout from the stages and mediator corrupted the FTXUI screen.

Original: Plain prints everywhere. Our first fix redirected cout inside the facade.

Change: A TLOG macro in Log.h prints in the test build and does nothing in the TUI build. The facade keeps its own activity log, and the History panel shows commands.

Why: Swapping a global stream is a hack, whereas a macro is simpler and needs no facade code.

8.Adapter

Problem: Different industries store information differently,according to formats, receipts might use JSON, HR records might use CSV and claims might be in excel format

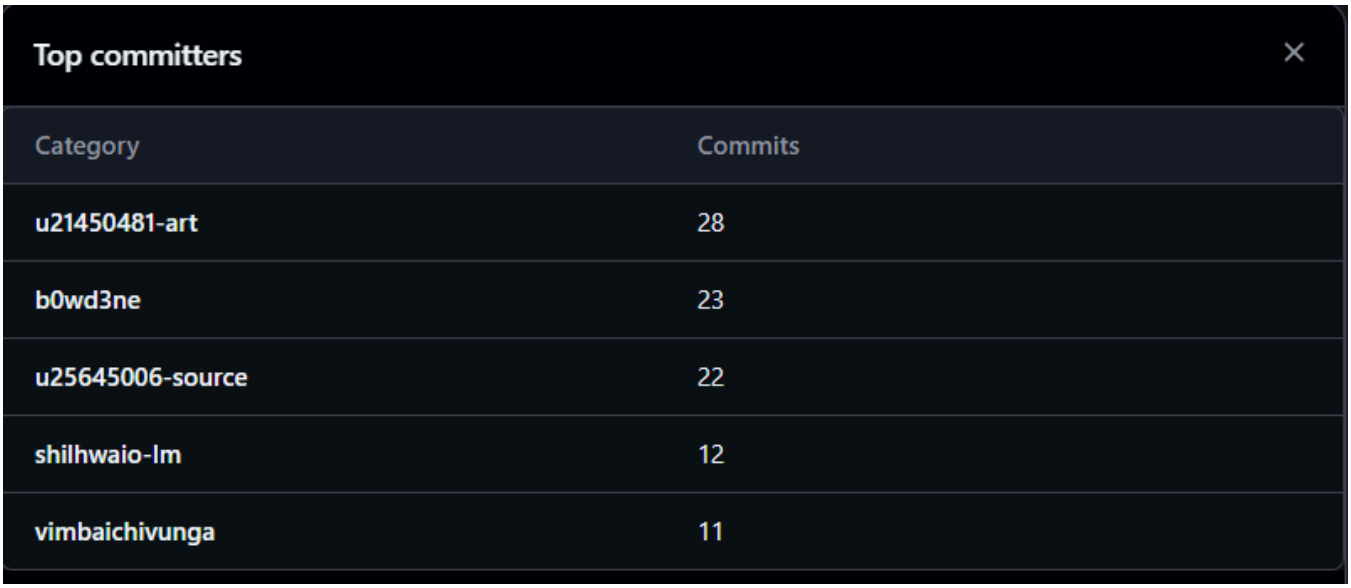
Original: we had a communication chanel but that took away the fact that TIVIDY had to be generic.

Change: Made a dataAdapter which adapts different formats of information into one TIVIDY can process.

Development practices

Git and Github

u21450481-art/214FinalProjectTIVIDY



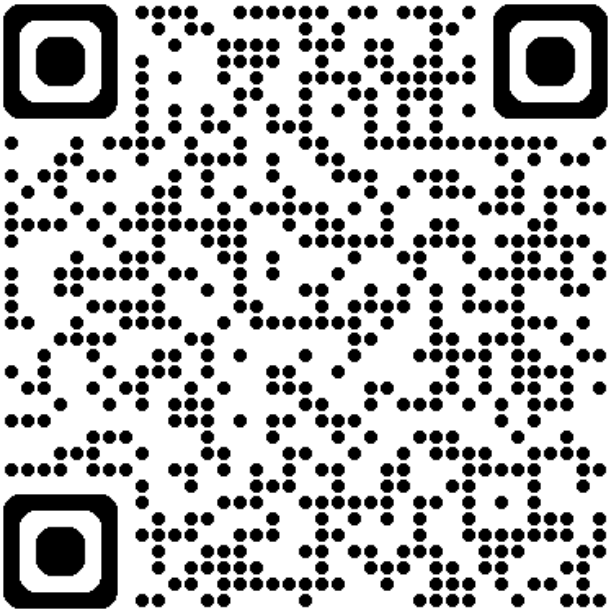
Top committers

Category	Commits
u21450481-art	28
b0wd3ne	23
u25645006-source	22
shilhwaio-lm	12
vimbaichivunga	11

To integrate changes safely, our repository uses two primary branches: main (for the final submission) and dev (for active development). Both branches are strictly protected by GitHub branch rulesets, meaning no one can push code directly to them.

When a team member works on a task, they create a separate feature branch. Once their code is working, they open a Pull Request (PR) to merge their changes into dev. This process helps us catch merge conflicts early and guarantees that our shared codebase always compiles cleanly before new features are added.

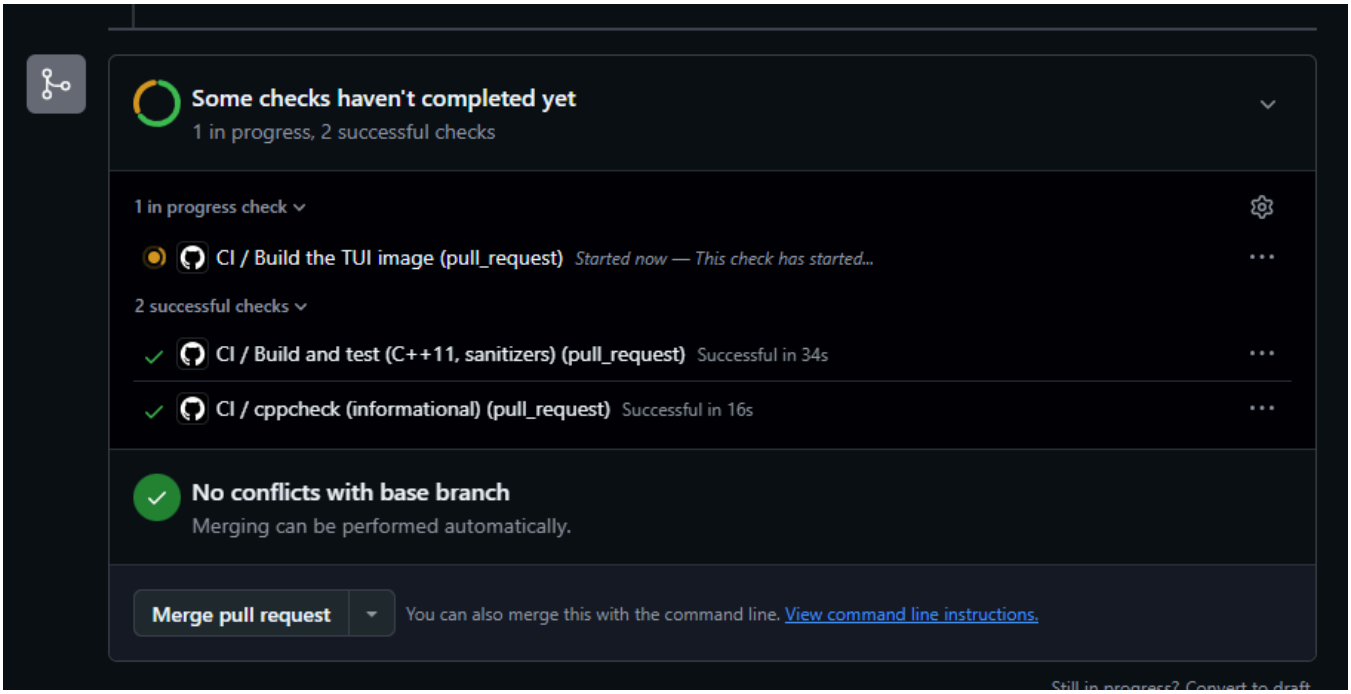
Because we communicate constantly as a team, we review these Pull Requests through direct discussion rather than using formal GitHub comment threads. Once all development is complete and tested on the dev branch, we will do one final merge from dev into main for our final project submission.



Bonus work

1. Docker
2. CI Pipeline

   Automated CI pipelines are configured to continuously build, test with sanitizers, perform static analysis via Cppcheck, and build container images on pull requests to ensure code quality and stability within every merge,which also reduces the chances of merge conflicts.



Build instructions

make : builds the TIVIDY project

make tui: runs the TUI for the workflow